AI8HT

ONE PLATFORM · DELIVERY PLAN

Parallel Build Plan & Work Allocation

Five people, five streams, and an application that is AI-ready before the GPU exists

Who works on what, in what order, and how five people stay out of each other's way in one codebase. The AI and LLM implementation is deliberately held until the GPU instance is provisioned — but every stream below is shaped so that connecting the model on that day is a configuration change and a worker deployment, not a rewrite.

Plan date **5 September 2026**	Team **5 — Divakar, Manisha, Navya, Chandra, Santhosh**	Baseline **Release e928d12**
Current host **VPS, moving to AWS t3.medium**	Target host **GPU instance, date TBD**	AI / LLM build **Held — seams built now**

SECTION 01

The situation, and the one constraint that shapes everything

This plan follows the Role Capability & Build-State Report (Rev 2). That report established what is live, what is a shell and what is unreachable. This one turns the backlog into five parallel streams and answers a harder question: how do we build for months without the GPU, and then switch the AI on in days rather than starting over?

Where we are

39
LIVE WORKFLOWS
Real API, database and mutations today

27
SCREENS TO BUILD
Shells and empty states for signed-in roles

45
BLOCKED SCREENS
Four roles nobody can sign into

5
DECISIONS PENDING
Product calls that gate real build work

The infrastructure position

The application runs today on the Plesk VPS and is being moved to AWS. The target instance has been resized from t3.small to **t3.medium** — 2 vCPU, 3.7 GB — which resolves the memory blocker identified during the migration review: the virus scanner alone peaks near 1.9 GB, and it cannot be omitted because the API refuses to start with document storage configured and no scanner. That box is a staging post. The application will later move to a **GPU instance**, which is what makes self-hosted inference possible.

> **The constraint: the GPU does not exist yet, and the team does.**
> Five people cannot wait for hardware. But building AI features against a model that is not there produces either stubs that get thrown away, or synchronous model calls wired into controllers that have to be torn out later. This plan takes a third route:
> **every artifact an LLM would eventually produce is modelled as a stored, reviewable row filled by a job.**
> Today those rows are filled by seeding, import or authoring. On GPU day, a generator worker starts filling the same rows through the same pipeline. No screen changes. No controller changes. No schema migration.

What "AI is held" means precisely

ITEM	STATUS	DETAIL
Model integration	HELD	No self-hosted model, no inference runtime, no GPU-dependent code is written until the instance exists.
Generated content	HELD	No LLM writes practice problems, resumes, prep material or explanations before GPU day.
Provider seam	BUILD NOW	The one place the application talks to a model becomes swappable by configuration.
Generation pipeline	BUILD NOW	Tables, event types, job states and review gates, exercised with non-AI content.
Grounding & retrieval	BUILD NOW	Deterministic and testable without any model at all.
Review & safety gates	BUILD NOW	Nothing generated reaches a user unreviewed — the pattern is built and used before it is needed.

> **One fact worth stating plainly before anyone plans around it.**
> The in-app assistant already exists and is well built — grounded retrieval over a curated corpus, a hardened system prompt, streaming responses, a daily per-user cap, and failure that closes rather than opens.
> **It is dark in production.**
> Its API key is not present in the server environment, so the service reports itself unconfigured and the widget returns unavailable. The team is not starting AI from zero; it is starting from a working seam that has never been switched on.

SECTION 02

The AI-readiness contract

Six rules. Every stream follows them from day one, and none of them requires a model to be present. If all five people hold to these, GPU day is a deployment; if any stream ignores them, GPU day becomes a refactor.

Rule 1 — An AI output is a row, never a response

Anything a model would eventually produce — a practice problem, a company preparation brief, a resume draft, a readiness explanation, a suggested intervention — is stored in a table before it is shown. Every such table carries the same five columns:

COLUMN	PURPOSE
`source`	`SEEDED`, `AUTHORED` or `GENERATED`. Today only the first two occur. The screen renders all three identically.
`generator_version`	Null until a model produces it. Afterwards, exactly which model and prompt version wrote this row.
`prompt_version`	Lets a bad prompt be traced to every row it produced, and those rows withdrawn.
`review_state`	`DRAFT` → `IN_REVIEW` → `PUBLISHED` → `WITHDRAWN`. Only published rows reach a user.
`generated_at`	Freshness, and the basis for regeneration policy.

This is not a new invention for this team. The assessment question bank already works exactly this way, with a reviewed state machine over a governed question pool. Reuse that shape rather than designing a second one.

Rule 2 — Generation is a job on the existing rail, not a call from a controller

No request handler ever waits on a model. The platform already runs an outbox and integration-event pipeline in production, with an EventBridge-to-SQS boundary defined in infrastructure and an automation worker mode configured. New event types are added now — `content.generation.requested`, `content.generation.completed` — and exercised by a non-AI worker that fills rows from seed data. On GPU day the consumer changes; the producer, the events and the screens do not.

> **Why this matters more than it looks.**
> A synchronous model call inside a controller ties response time to inference time, makes every screen fail when the GPU is busy, and cannot be rate-limited per tenant or retried. Putting generation on the queue from the start means a slow or absent model degrades to *content not ready yet* — which is exactly what the screens already render as NA today.

Rule 3 — One provider port, swappable by configuration

The assistant service already accepts its API key, model name and HTTP implementation by injection, and exposes a `configured` flag that fails closed. Two additions make it GPU-ready: a **base URL** so it can point at a local inference endpoint instead of a public API, and a thin **provider adapter** so an OpenAI-compatible server — which is what vLLM, TGI and Ollama all expose — is selected by config. Nothing else in the application should ever know which model is answering.

Rule 4 — Retrieval and grounding stay deterministic

The existing retrieval is lexical, deterministic and unit-tested without any network call. Keep that property. Grounding selection, prompt assembly and safety boundaries must all be testable on a laptop with no GPU and no API key. The model is the only non-deterministic part of the system, and it must be the only part.

Rule 5 — Nothing generated reaches a user unreviewed

Every generated row lands in `DRAFT`. A human moves it to `PUBLISHED`. This is already how assessment questions work and it is the difference between a feature that can be launched to students and one that cannot. Build the review queue during the hold, using seeded content, so it is proven before a model ever writes to it.

Rule 6 — Every AI feature ships behind a flag that defaults to off

Per-feature, not one global switch. The assistant is the proof this works: it is unconfigured in production right now and the application is entirely healthy — the widget simply reports itself unavailable. Every future AI feature inherits that behaviour, plus a cost ceiling and a per-user cap of the kind the assistant already enforces.

> **The test for every design decision during the hold.**
> Ask: *"if a GPU appeared tomorrow, what would I have to change?"* If the answer includes a schema migration, a screen change or a controller change, the design is wrong. The only acceptable answers are: deploy a worker, set a configuration value, turn on a flag.

SECTION 03

Five streams, chosen to not collide

The allocation is by **vertical slice** — each person owns a domain from its database table through its API controller to its screens. This is deliberate: the alternative split, by layer, would put all five people in the same three files. Everyone here is equally capable, so the streams are sized by surface area rather than by difficulty.

STREAM	OWNER	DOMAIN	DEPENDS ON
A · Platform	Chandra	AWS migration, host hardening, CI, observability, GPU preparation	Nothing
B · Hiring	Manisha	Employer and student hiring: interviews, offers, pipeline completion	Nothing
C · Institution	Navya	Institution staff actions, interventions, reports, staff lifecycle	Nothing
D · Learner	Santhosh	Student experience, readiness presentation, and the content-generation pipeline every AI feature will use	Nothing
E · Core	Divakar	Architecture, the AI seam, product decisions, review and release	Unblocks B, C, D

Four of the five streams start with no dependency on any other. That is the point: nobody waits in week one.

Chandra

Stream A · Platform, AWS and GPU readiness

Why this split

This stream touches no application source at all, which makes it perfectly parallel — it can run at full speed from hour one without ever waiting for or blocking a code review. It is also the stream that determines whether GPU day is smooth.

Scope

- **Complete the VPS to AWS migration** onto t3.medium: PostgreSQL 16 from the PGDG repository to match the source cluster exactly, nginx, certbot, ClamAV with its scheduled signature updates enabled at boot, the four systemd units and their timers.
- **Rotate the exposed secrets** — the object-storage backup key first, then the database password, the monitoring ping URLs and the automation signing secret. Carry the file-encryption key verbatim; it cannot be rotated without re-encrypting stored documents.
- **Move builds off the application host.** Two vCPUs should not be running a production build while serving traffic. Extend the existing CI workflows to produce the release artifact.
- **Add swap and monitoring headroom** on t3.medium; there is none today.
- **GPU preparation** — the deliverable that matters most: instance family and driver plan, an inference runtime choice that exposes an OpenAI-compatible endpoint, a private network path so the model is never publicly reachable, model storage sizing, and a written cost ceiling with an idle-shutdown policy.
- **DNS cutover** for one.ai8ht.pro with a lowered TTL, and a rollback window before the VPS is decommissioned.

Owns: `deploy/` , `.deploy/` , `infra/` , `.github/workflows/` , server configuration. **Touches no application code.**

First task

Rotate the backup storage key. It grants write and delete on the off-site backups and it has been disclosed. Everything else waits behind that.

Done when

one.ai8ht.pro serves from AWS with verified backups, working monitoring and a tested rollback; and a one-page GPU provisioning runbook exists that Divakar can execute the day the instance is approved.

Manisha

Stream B · Hiring — employer and student

Why this split

Hiring is the most complete backend in the product and the most under-exposed. Eleven endpoints are implemented and enforced; the interface reaches them through two screens. This stream converts already-paid-for work into visible product, and it is almost entirely self-contained in files nobody else needs.

Scope

- **Week one, no backend:** re-point the Recruiter's Interviews and Offers sidebar items at a filtered Applicants view. Two screens stop claiming “not built” about features that work.
- **Add the missing read paths.** Interview create, state change and outcome all exist; nothing lists interviews. Offer extend and withdraw exist; nothing lists offers. Add both list endpoints.
- **Build the Employer Interviews and Offers views,** surfacing interview state, interview outcome and offer withdrawal — three implemented mutations with no interface anywhere today.
- **Student Interviews:** add interview fields to the student application record, then the student-facing view. Employers schedule real interviews the student currently cannot see.
- **Replace two placeholders sitting inside a live workflow:** the scheduler hardcodes a one-hour video slot at 10:00 one week out, and the offer action collects compensation through a browser prompt dialog. Production holds zero interviews and zero offers, so there is nothing to regress.
- **Talent Pool** as a real saved pool rather than an alias of the applicant list.

Owns: `employer-hiring.controller.ts`, `student-hiring.controller.ts`, `packages/hiring/`, `hiringPages.tsx`, the employer branch of the portal dispatcher. **Migration block:** 0016–0019.

First task

The sidebar re-point. It is hours of work, needs no backend, and removes a false statement the product currently makes to every recruiter.

Done when

A recruiter can run a hire end to end from dedicated screens — post, publish, shortlist, schedule, record an outcome, offer, withdraw — and the student sees their interviews and responds to offers.

Navya

Stream C · Institution staff

Why this split

Twenty-one read-only screens across Faculty and Department Head are one missing concept wearing five costumes. This stream builds that one concept and lights up four roles at once — the highest leverage single table in the backlog, and the same table a folded-in Career Counselor would need.

Scope

- **The staff-action record.** One table with a type discriminator, not five feature-specific tables: *actor, subject student, type, state, note, due date, audit trail.* It serves Interventions, Assigned Tasks, Faculty Notes, Academic Verification and Meetings.
- **Make Interventions real** for all four institution staff roles. The screen is already a pilot-exact shell, and its not-attempted queue is already live data — keep that and fill the rest.
- **Wire Institution Reports.** The career-readiness report endpoint exists and is callable; the screen simply never calls it. Pure wiring, and a quick early win.
- **Complete the staff lifecycle.** Invitation works end to end; revoke, resend and end-membership need an endpoint and a control. The screen already names its own gap.
- **Placement Phase 1** (from week five): promote Students, Departments, Batches, Student Search and Readiness from the demo build to the real Placement Officer portal, served by institution endpoints the Administrator already uses. Three screens becomes eight with no new tables.

Owns: a new staff-actions controller, the institution portion of the onboarding controller, the institution branch of the portal dispatcher, `staffPages.tsx`, `staffConfig.ts`, the Interventions page. **Migration block:** 0020–0023.

First task

Institution Reports wiring — hours, no new backend — then design the staff-action table and circulate the schema before writing the migration, because Career Counselor folding depends on getting its shape right.

Done when

Faculty and Department Head can each record something a colleague can see and act on, and the Placement Officer portal is no longer three screens.

> **Held pending a decision from Divakar.**
>
> Placement Phase 2 — pipeline, applications, employer engagement — requires the institution to read employer-side application data, and no such path exists in any controller. That is a consent and disclosure decision, not a schema. Do not start it before the rule is written down.

Santhosh

Stream D · Learner experience and the generation pipeline

Why this split

This is the stream that makes the GPU worth having. Five student screens — Practice, Company Prep, Career Tools, Scholarships, Resume Builder — are pilot-exact shells whose eventual content source is an LLM. Building their **tables, contracts and review pipeline now**, filled by seeding, means GPU day switches on five features rather than starting them. This stream owns the AI-readiness contract in practice while the AI itself stays held.

Scope

- **Readiness presentation first.** Replace the NA/100 ring with a band ring: four segments, the current advisory band highlighted, existing readiness factors as the legend, and the four category rings driven by the per-topic mastery projections that already exist. The engine has already run — this is presentation, not computation, and it lands on the student dashboard and all four staff dashboards at once.
- **Build the content-generation pipeline** to the six rules in Section 02: content tables carrying source, generator version, prompt version, review state and timestamp; the two generation event types on the existing outbox rail; a review queue modelled on the assessment question bank.
- **Fill the five shells with non-AI content.** Seeded and imported practice problems, company briefs, scholarship records, resume templates, career tools. Modest volume is fine — the point is that the pipeline is exercised end to end before a model touches it.
- **Student Experience records** — projects, internships, portfolio. The one student screen with no shell at all.
- **The AI Career Coach page,** built over the assistant that already works as a widget.

Owns: `pilotParityPages.tsx` (the five student pages), `studentDashboardV3.tsx`, `careerReadinessApiPanel.tsx`, `studentPages.tsx`, a new content controller and content package. **Migration block:** 0024–0027.

First task

The readiness band ring. It is visible, it is quick, it corrects the product's most conspicuous NA, and it requires nothing from anyone else.

Done when

No readiness ring anywhere says NA; all five shells serve real content through a pipeline whose only missing participant is the generator.

> **Held.**
> No prompt engineering, no model calls, no generated content. If a task cannot be completed without a model, it belongs in Phase 3 — write the contract and move on.

Divakar

Stream E · Core, AI seam and delivery

Why this split

Three streams are blocked on decisions only the product owner can take, and one seam — the model provider port — determines whether GPU day costs a day or a quarter. Both belong here. This is not a coordination role: the AI seam, the shared-file surgery and the release path are substantial engineering.

Scope — week one, because others are waiting

- **The four blocked roles.** Recommended: enable Interviewer scoped to three screens; fold Hiring Manager into Recruiter; fold Career Counselor into institution staff; drop Institution Analyst as a role and make analytics-and-export a permission. This turns 45 unreachable screens into roughly three to build.
- **The institution disclosure rule** — what a college may see about its own student's application to an employer, and on whose consent. Navya's Placement Phase 2 cannot start without it.
- **Sign off the readiness presentation** so Santhosh can start: band ring, or a numeric projection that would fight a deliberate database constraint. The recommendation is the band.
- **Platform Super Admin and Security Admin** — give each a distinct remit or retire it.

Scope — the AI seam, built during the hold

- **Add a base URL and provider adapter** to the assistant service. It already injects key, model and HTTP implementation and fails closed when unconfigured; these two additions make an OpenAI-compatible local endpoint a configuration value.
- **Move the rate limiter out of process memory.** The daily per-user cap currently lives in a map inside one process; it will not survive a restart or a second instance.
- **Own the grounding corpus and prompt governance** — versioned prompts, the hard boundaries already in the system prompt, and the rule that generated content is traceable to the prompt that produced it.
- **Write the GPU cutover runbook** jointly with Chandra, so switching on is executed rather than improvised.

Scope — keeping four streams parallel

- **The Phase 0 file surgery** in Section 05: splitting the three hot files before anyone else touches them.
- Review every pull request, hold the migration-number registry, cut releases, and keep the reality matrix current.

Owns: `apps/api/src/assistant/`, `App.tsx`, `api.ts` structure, `packages/authorization/`, the membership role trigger migration, the migration registry. **Migration block:** 0016 reserved for the role trigger, then 0028+.

First task

Write the four role decisions down and circulate them. Three people are shaping work around answers that do not exist yet.

Done when

No stream is blocked on a decision; the assistant can be pointed at a local endpoint by changing configuration alone; and the cutover runbook is written and rehearsed.

> **On seniority.**
> All five are equally capable, so nothing here is graded by difficulty. The streams differ in *surface area and dependency*, not in skill. Chandra's stream is isolated because infrastructure work benefits most from never waiting on review. Divakar's carries the decisions because three other streams are blocked on them, and the person who can unblock should not also be on the critical path of a feature.

SECTION 04

Keeping five people out of one another's way

Vertical slices remove most collisions, but three files in this codebase are touched by everything, and migration numbers collide by nature. These rules are what make the parallelism real rather than aspirational.

Phase 0 file surgery — before anyone starts a feature

Three files are unavoidable contention points. Divakar splits them in week one, alone, in a single small pull request that everyone rebases onto. Doing this first costs a day; not doing it costs a merge conflict every week for three months.

FILE	LINES	SPLIT INTO
`rolePortalPages.tsx`	837	Three modules by portal family — institution, employer, operations — so Navya and Manisha never edit the same file. The shared dispatcher stays small and belongs to Divakar.
`api.ts`	974	Per-domain client modules re-exported from one index, so each stream appends to its own file. This is the single highest-collision file in the repository — every stream adds functions to it.
`pilotParityPages.tsx`	344	Move the institution Interventions page out to its own module. It is Navya's; the five student pages are Santhosh's. Today they share one file.

Migration number blocks

Migrations are numbered sequentially and two people creating `0016` in the same week is a guaranteed conflict. Blocks are reserved up front; nobody takes a number outside their block without asking Divakar.

OWNER	BLOCK	EXPECTED USE
Divakar	0016	Membership role trigger, once the role decisions are made
Manisha	0017–0019	Interview and offer read models, talent pool
Navya	0020–0023	Staff-action record, staff lifecycle
Santhosh	0024–0027	Content tables, generation jobs, experience records
Divakar	0028+	Anything cross-cutting, allocated on request

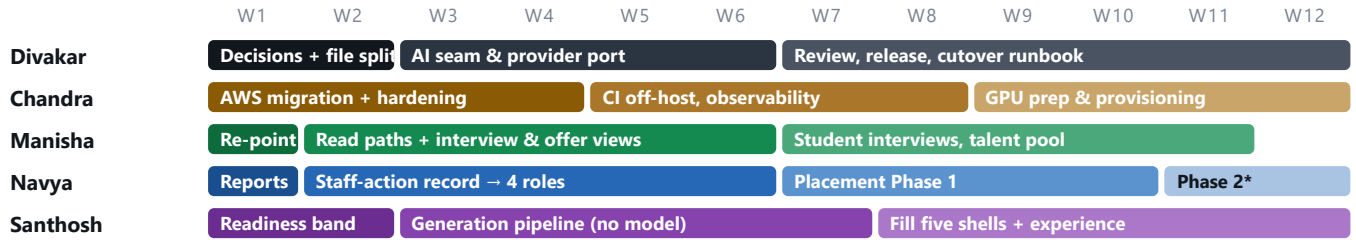
Working agreement

- **One branch per stream, rebased on `main` daily.** Long-lived branches are how a five-person team discovers a month's worth of conflict at once.
- **Merge small and often.** A stream that has not merged in a week is a stream that is about to hurt someone.
- **Never reformat a file you do not own.** A whitespace pass across a shared file turns a clean merge into a manual one.
- **Schema before migration.** Circulate a table design before writing the migration — especially the staff-action record, which three roles depend on.
- **The reality matrix is updated by whoever changes reality.** It is already a release behind; that is how it got there.
- **Tests run before merge.** The workspace already has typecheck and test scripts and two CI workflows; keep them green rather than reviving them later.

SECTION 05

Twelve weeks, five streams, in parallel

Weeks are indicative and deliberately not dated — the shape and the ordering matter more than the calendar.
Note that nobody is idle in week one, and no stream waits on another to begin.



* Placement Phase 2 starts only once the institution disclosure rule is written. The GPU bar is provisioning readiness, not a promise of hardware.

What each phase delivers

PHASE	WEEKS	OUTCOME
0 · Unblock	1	Decisions written, hot files split, three visible wins shipped with no backend work: the Interviews and Offers re-point, Institution Reports wiring, and the readiness band started.
1 · Verticals	2–7	Hiring becomes navigable; one staff-action table lights up four institution roles; the generation pipeline exists and is exercised without a model; the application runs on AWS.
2 · Depth	8–12	Student interviews and talent pool; Placement Officer at eight screens; five student shells serving real seeded content through the review queue; GPU runbook rehearsed.
3 · GPU day	On arrival	Deploy the generator worker, point the provider port at the local endpoint, turn flags on one feature at a time. No schema, screen or controller changes.

SECTION 06

GPU day: the cutover checklist

The measure of this plan is how short this list is. If the six rules in Section 02 have been followed, switching the AI on is a deployment and a set of configuration values — not a project.

What must already be true before the instance is provisioned

OWNER	PRECONDITION	VERIFICATION
Divakar	Provider port accepts a base URL	The assistant answers against a local stub endpoint with no code change — only configuration.
Divakar	Rate limiting survives a restart	The daily cap is enforced from shared storage, not process memory.
Santhosh	Content tables carry generation columns	Every content row already has source, generator version, prompt version and review state — populated as SEEDED.
Santhosh	Generation events flow end to end	A non-AI worker consumes a generation request and writes a DRAFT row that a reviewer publishes.
Santhosh	Review queue is in use	Reviewers have already published seeded content through it. The queue is proven before a model uses it.
Chandra	Private network path	The inference endpoint is reachable from the API host and from nowhere else. Never a public port.
Chandra	Cost ceiling and idle policy	A written, agreed spend limit and an automatic shutdown rule for an idle GPU.
Chandra	Model storage sized	Weights, cache and logs have somewhere to live that is not the application volume.

The cutover itself

#	STEP	NOTES
1	Provision the GPU instance and inference runtime	Serving an OpenAI-compatible endpoint on the private network.
2	Point the provider port at it	Base URL and model name in server configuration. No deployment of application code.
3	Enable the assistant for a small group	It is already built, already grounded, already capped. This is the safest first AI feature to switch on and the fastest to evaluate.
4	Deploy the generator worker	It consumes the generation events that have been flowing since Phase 1 and writes DRAFT rows.
5	Generate into review, one feature at a time	Practice first — it is the most structured and easiest to judge. Nothing reaches a student unreviewed.
6	Raise flags feature by feature	Each with its own cap. Never a single global AI switch.

> **Start with the assistant, not with generation.**
> It is finished, grounded, boundaried and capped, and it fails closed. Pointing it at the new endpoint is the cheapest possible real test of the GPU, the runtime and the network path — and if the model quality disappoints, you find out on a low-stakes help widget rather than on content shown to students.

SECTION 07

What would break this plan

RISK	LIKELIHOOD	MITIGATION
Decisions do not arrive in week one	High	The single largest risk. Three streams shape work around four role decisions, a disclosure rule and a readiness sign-off. Divakar's week one is these, and nothing else. If they slip, streams C and D drift into work that may be discarded.
Someone builds a synchronous model call	Medium	It is the intuitive way to add AI and it is the one thing that makes GPU day a refactor. Section 02 exists to prevent it; review enforces it.
The hot files are not split first	Medium	Every stream edits the API client. Without the Phase 0 split, weekly merge conflicts in a 974-line shared file become the team's dominant cost.
The GPU arrives earlier than expected	Medium	A good problem, handled by ordering: the pipeline is built before the shells are filled, so an early GPU simply moves generation forward. Nothing has to be undone.
t3.medium proves tight	Medium	3.7 GB with the scanner resident and no swap is workable but not generous. Add swap early, and keep builds off the host — both are in Chandra's scope.
Placement Phase 2 starts before the disclosure rule	Low	It would produce a data path that has to be withdrawn. Explicitly gated in Navya's scope.
Generated content reaches a student unreviewed	Low	Prevented structurally: rows land in DRAFT and only PUBLISHED rows are served. Built and exercised long before a model exists.

Scope and status of this plan

This plan is derived from a source-and-database audit of release e928d12; no application code has been changed in producing it, and the allocation reflects the codebase as it stands rather than as it is documented. Weeks are indicative. The five names are assigned to streams by surface area and dependency, not by difficulty — the team is described as equally capable and the plan assumes it.

> **The single sentence to hold on to.**
> Build every AI-shaped feature as a table filled by a job behind a review gate, and the day the GPU arrives you will deploy a worker and change a configuration value — instead of discovering that the application was never shaped to accept a model at all.